NDEJJE UNIVERSITY

FAULTY OF SCIENCE AND COMPUTING AND DEPARTMENT OF SCIENCE, SPORTS SCIENCE AND IT, AND DEPARTMENT OF COMPUTING.

COUSREUNIT: CRYPTOGRAPHY & CYBER SECURITY

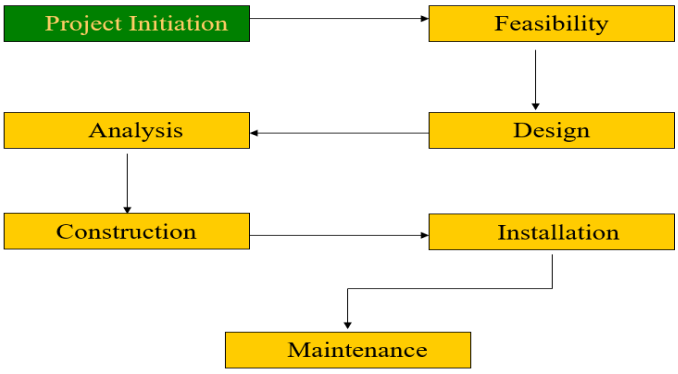
LECTURER: MR. MUCHAKE BRIAN

COUSRECODE: BCS 3203

NAME:	**MUWONGE DERRICK**
REG NO:	**23/2/306/D/063**

Software Development Life Cycle (SDLC) Models & Quality Practices

The Software Development Life Cycle (SDLC) is a structured framework used by software engineers to design, develop, test, deploy, and maintain software systems. It provides a systematic approach that helps teams build high-quality software while meeting user requirements within time and budget constraints.



In software development, projects often involve multiple stakeholders such as developers, testers, project managers, system analysts, and clients. Each stakeholder plays a different role in ensuring that the final software product functions correctly and meets business objectives.

Without a clear development process, software projects may encounter several problems such as:

Poor system quality

Missed deadlines

Budget overruns

Miscommunication between teams

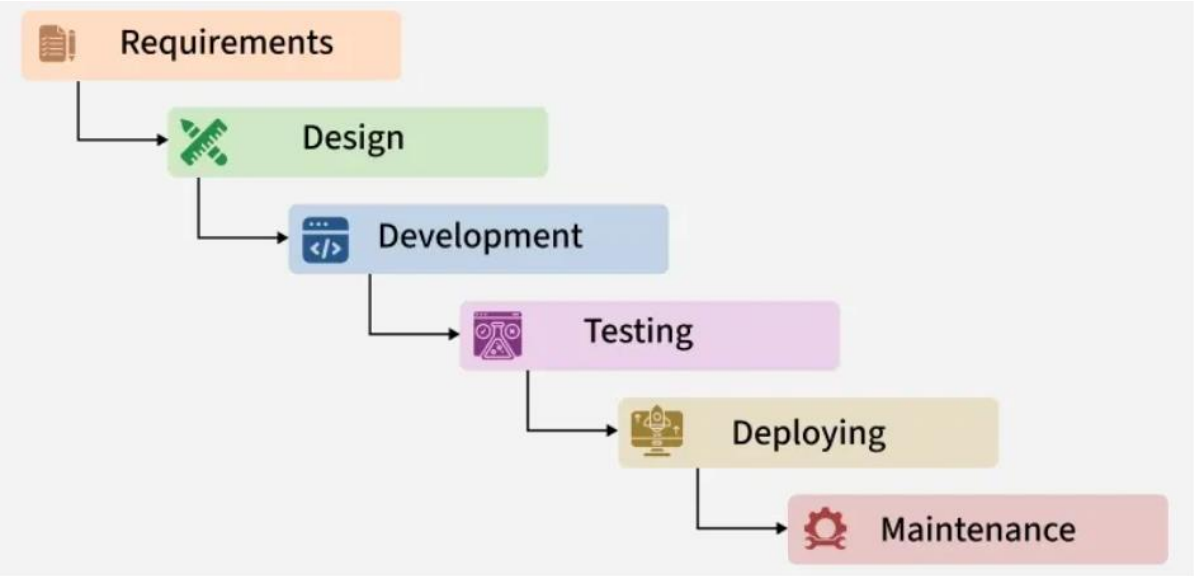
Failure to meet user requirements

SDLC models provide organized phases and development strategies that guide teams throughout the entire software development process. These models help teams plan activities, manage risks, maintain quality, and deliver reliable software products.

Different SDLC models exist because software projects differ in complexity, size, and requirements. Some models focus on strict planning and documentation, while others emphasize flexibility, customer involvement, and rapid delivery.

1. Waterfall Model

The Waterfall Model is a traditional software development methodology that follows a linear, phase-by-phase approach, where each phase must be completed before moving to the next. It does not allow backtracking and permits only minimal changes once a phase is completed. The model is simple, well-structured, and easy to manage, offering high predictability and clearly defined milestones throughout the development process.



Features of the Waterfall Model

1. Sequential Approach
   Development follows a linear, step-by-step process where each phase is completed before moving to the next.
2. Document-Driven Process
   Detailed documentation is created at every stage to clearly define requirements, design, and progress.
3. Quality Control
   Strong emphasis is placed on verification and testing to ensure each phase meets its defined requirements.
4. Detailed Planning
   The project scope, schedule, and deliverables are carefully planned and monitored throughout the lifecycle.

Phases of the Waterfall Model

1. Requirements Analysis

This is the first and most critical stage of the development process. During this phase, developers and system analysts gather information about the system requirements from clients and stakeholders.

Activities performed in this phase include:

Conducting interviews with users

Studying existing systems

Analyzing business processes

Identifying functional and non-functional requirements

The final output of this stage is a document known as the Software Requirements Specification (SRS). This document describes the system’s expected functionality, performance requirements, and constraints.

2. System Design

After the requirements have been clearly defined, the next step is designing the system architecture. This phase describes how the system will be built to satisfy the requirements.

Design activities may include:

Designing system architecture

Creating database schemas

uDesigning user interfaces

Developing system models and diagrams

Defining hardware and software requirements

The design stage acts as a blueprint that guides developers during implementation.

3. Implementation (Coding)

During this phase, developers begin writing the actual program code according to the design specifications created earlier.

The software is usually divided into smaller components called modules. Each module is coded separately and later integrated into the complete system.

Programming languages such as Java, Python, C#, JavaScript, or PHP may be used depending on the system requirements.

4. Testing

After the software has been developed, it must be tested thoroughly to ensure that it functions correctly and meets the requirements specified in the SRS document.

Testing helps identify errors, bugs, and system failures before the software is delivered to users.

Common types of testing include:

Unit Testing – testing individual modules

Integration Testing – testing interactions between modules

System Testing – testing the entire system

User Acceptance Testing (UAT) – ensuring the system meets user expectations

5. Deployment

Deployment is the stage where the finished software product is delivered and installed in the user's environment. At this point, users begin interacting with the system.

Deployment may involve:

Installing the system on servers

Configuring databases

Training users

Providing system documentation

6. Maintenance

After deployment, the software enters the maintenance phase, which may last for many years.

Maintenance includes:

Fixing bugs discovered after release

Updating the system to meet new requirements

Improving system performance

Adapting the system to new technologies

Maintenance is often the longest phase in the software lifecycle.

Advantages of the Waterfall Model

Simple and easy to understand.

Clear documentation at every stage.

Easy project management due to defined milestones.

Disadvantages

Very difficult to accommodate requirement changes.

Late detection of problems since testing occurs near the end.

Not suitable for large or complex projects.

2. V-Model (Verification and Validation Model)

The V-Model is an extension of the Waterfall Model that emphasizes testing and quality assurance at every stage of development.

The name comes from the V-shaped structure that represents the relationship between development phases and testing phases.

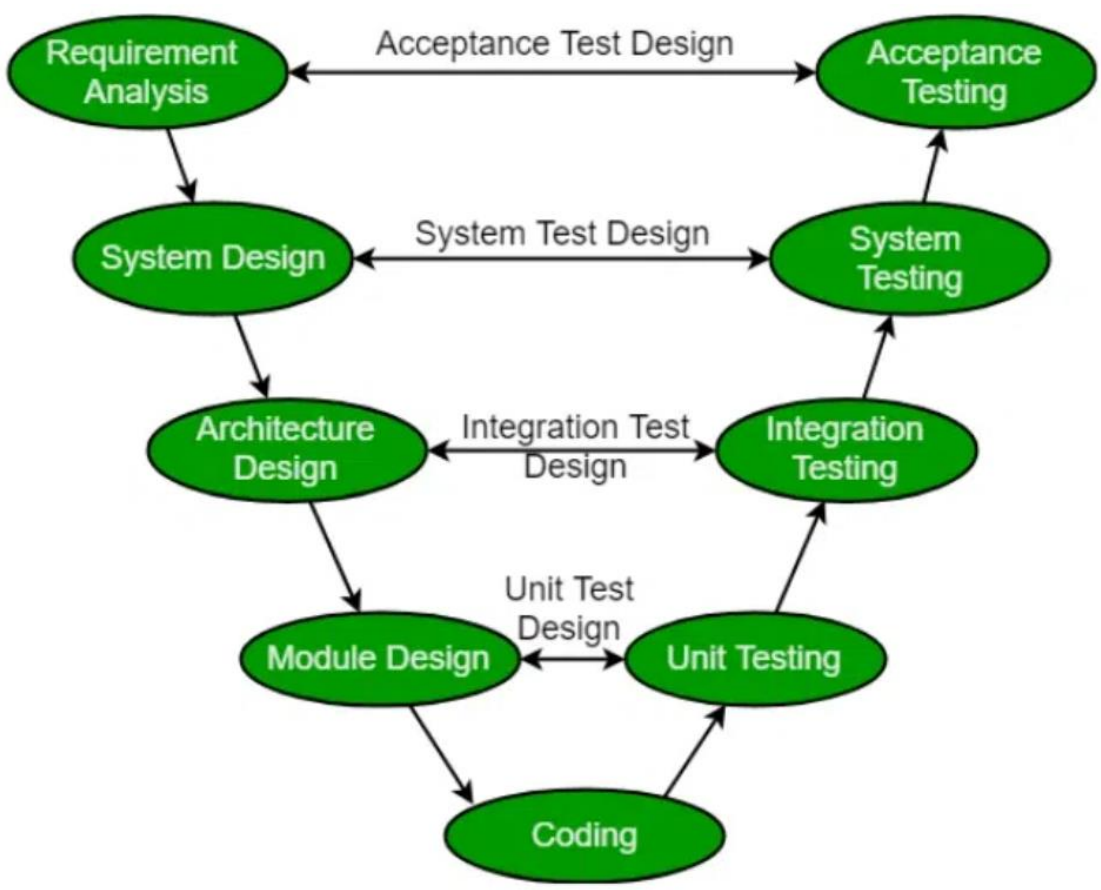
The left side of the "V" represents development activities, while the right side represents corresponding testing activities.

Verification vs Validation

Verification: Ensuring the software is built correctly according to specifications.

Validation: Ensuring the final software meets the needs of the users.

Development Phases



Requirements Analysis

Defines system requirements.

These requirements are later verified through acceptance testing.

System Design

Defines the overall system architecture and structure.

Architectural Design

Defines system modules and interactions between components.

Module Design

Testing Phases

Each development phase has a matching test Specifies detailed design of individual modules.

Coding

Implementation of the software system.

phase.

Unit Testing

Individual modules are tested separately to ensure they work correctly.

Integration Testing

Modules are combined and tested to verify that they interact properly.

System Testing

The entire system is tested to ensure it meets functional and performance requirements.

Acceptance Testing

The client or end users test the system to verify that it satisfies their needs.

Advantages

- Early planning of testing activities
- Higher product quality
- Clear relationship between development and testing

Disadvantages

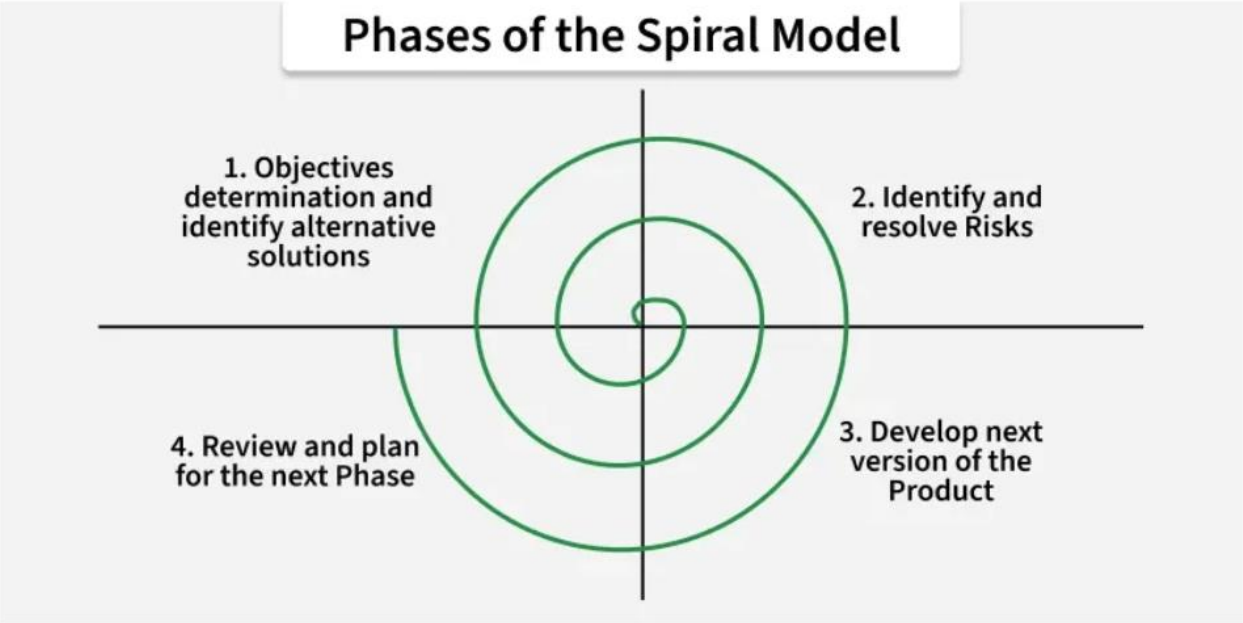
- Not flexible for requirement changes
- More rigid compared to Agile methods

3. Spiral Model

The Spiral Model is a risk-driven development model that combines iterative development with systematic risk analysis.

It was introduced by Barry Boehm in 1986 and is particularly useful for large, complex, and high-risk projects.

In the Spiral Model, development progresses through repeated cycles (spirals). Each cycle includes planning, risk analysis, development, and evaluation.



Main Phases of Each Spiral Loop

Planning

Objectives and requirements are identified for the next development stage.

Risk Analysis

Potential risks are identified and evaluated. Strategies are created to reduce or eliminate these risks.

Examples of risks include:

Technology limitations

Budget constraints

Security vulnerabilities

Project delays

Engineering

Developers design, build, and test the software components.

Evaluation

The system is evaluated by stakeholders and customers who provide feedback for the next development cycle.

Advantages

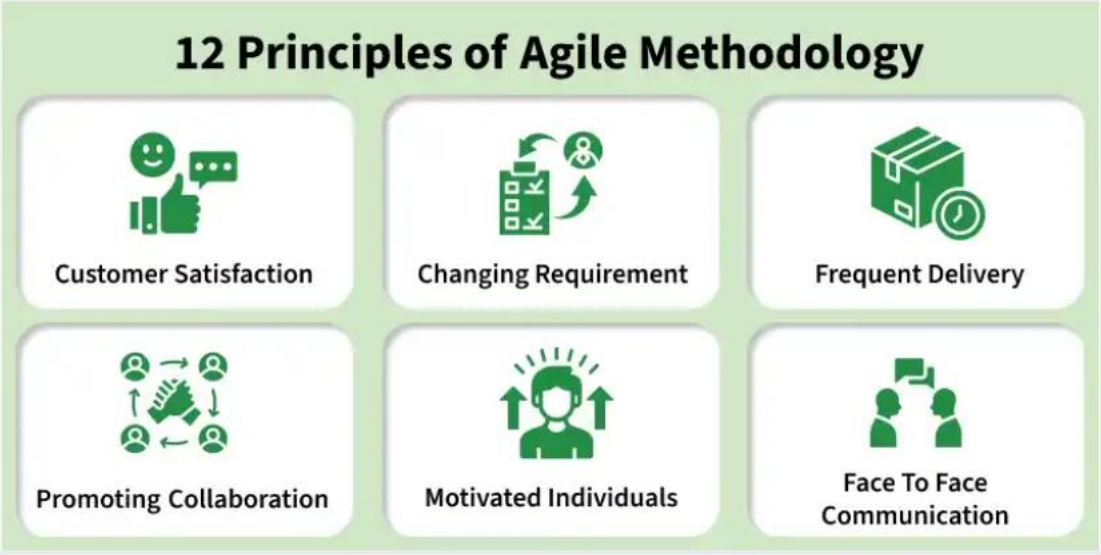
- Excellent risk management
- Continuous customer feedback

- Suitable for large and complex projects

Disadvantages

- Expensive and complex to manage
- Requires experienced project managers
- Not suitable for small projects

4. Agile Methodology



Agile is a modern software development methodology that focuses on flexibility, collaboration, and incremental delivery of software.

Unlike traditional models that require detailed planning before development begins, Agile allows teams to adapt to changes throughout the development process.

Agile became popular after the publication of the Agile Manifesto in 2001.

Agile Development Process

Agile divides development into small time-boxed cycles called Sprints, which usually last 2 to 4 weeks.

During each sprint:

The team plans the features to be developed.

Developers design and implement those features.

Continuous testing is performed.

The working product is demonstrated to stakeholders.

Each sprint produces a working version of the software called an increment.

Popular Agile Frameworks

Scrum

Uses defined roles such as Product Owner, Scrum Master, and Development Team.

Kanban

Uses visual boards to track workflow and manage tasks.

Extreme Programming (XP)

Focuses on strong coding practices such as pair programming and test-driven development.

Advantages

Highly flexible and adaptable

Faster delivery of functional software

Strong customer involvement

Disadvantages

Requires strong team collaboration

Difficult to predict project timelines

Less emphasis on documentation

5. DevOps Quality Practices

DevOps is a cultural and technical approach that integrates software development and IT operations.

Traditionally, developers built software while operations teams handled deployment and system management. This separation caused delays and communication problems.

DevOps promotes collaboration, automation, and continuous improvement.

Key DevOps Practices

Continuous Integration (CI)

Developers frequently integrate their code changes into a shared repository.

Continuous Delivery (CD)

Software updates are automatically prepared for release.

Infrastructure as Code (IaC)

System infrastructure is managed through configuration files and scripts.

Automated Testing

Testing processes are automated to detect issues quickly.

Monitoring and Logging

Applications are monitored continuously to detect failures and performance problems.

Benefits of DevOps

- Faster software releases
- Improved collaboration
- Better system reliability
- Faster problem detection

6. Quality in Continuous Integration and Continuous Delivery (CI/CD)

CI/CD practices automate the process of building, testing, and deploying software.

These practices help maintain consistent software quality while allowing frequent updates.

Continuous Integration (CI)

Continuous Integration requires developers to merge their code into a shared repository frequently.

Each code integration triggers automated actions such as:

Building the application

Running automated tests

Performing code quality checks

This helps detect issues early in development.

Continuous Delivery (CD)

Continuous Delivery ensures that the software can be released to production at any time.

Deployment pipelines automatically move the software through stages such as:

Build

Test

Quality verification

Staging environment deployment

Production deployment

Benefits of CI/CD

- Faster delivery of new features
- Reduced deployment risks
- Improved software quality
- Less manual work

Conclusion

The Software Development Life Cycle provides structured models for managing software development projects effectively. Traditional models such as Waterfall, V-Model, and Spiral focus on structured development and documentation, while modern approaches like Agile emphasize flexibility and rapid delivery.